Hướng dẫn nắm website và thuyết trình theo rubric

Tài liệu này cho bạn biết **website làm gì, chạy thế nào, và từng tiêu chí chấm điểm cần bạn hiểu / nói gì**. Đọc theo thứ tự: phần 1 và 2 để nắm tổng quan, phần 3 là trọng tâm (từng tiêu chí), phần 4 là kịch bản demo, phần 5 là câu hỏi thường gặp.

> Mọi tên tệp và hàm trong tài liệu đều có thật trong dự án, bạn có thể mở ra và chỉ cho giảng viên. Muốn hiểu sâu **logic xử lý từng tính năng** (đặc biệt là chặn spam, công thức giá, luồng gửi báo giá, so sánh sản phẩm) thì đọc thêm LOGIC-XU-LY.md. Tài liệu này là bản tóm tắt để thuyết trình, tài liệu kia là bản giải thích chi tiết.

---

1. Website này là gì (nói được trong 30 giây)

Ericss Furniture là website của một xưởng nội thất ngoài trời cao cấp (teak, nhôm, dây thừng, mây nhựa). Khách xem giới thiệu và quy trình, duyệt 26 sản phẩm (có thể **so sánh tối đa 3 mẫu**), ước giá, rồi **gửi yêu cầu báo giá kèm MOQ (số lượng đặt tối thiểu) của chính họ**. Yêu cầu được lưu vào **Google Sheet**, khách nhận email cảm ơn và mã QR. Website là sản phẩm của **bài thi** môn Thiết kế Web; Ericss Furniture là thương hiệu giả định.

Đây là website **tĩnh** (HTML, CSS, JavaScript thuần, đặt trên GitHub Pages), không có máy chủ riêng. Mọi "logic" nằm ở JavaScript trong trình duyệt:

- dữ liệu sản phẩm và bảng giá nằm trong các bảng dữ liệu có **cấu trúc JSON** (tệp `data/*.js`);
- JavaScript đọc các bảng đó, vẽ giao diện, tính tiền;
- dữ liệu của khách (giỏ, yêu thích, lịch sử) lưu bằng `localStorage`;
- yêu cầu báo giá gửi lên Google Sheet qua **Google Apps Script**.

Bốn trang

Trang	Khách làm gì
`index.html` (trang chủ)	Xem hero, băng số liệu, bốn chất liệu, quy trình 8 bước, thư viện ảnh; bấm ảnh để **xem lớn (modal)**; nhảy giữa các phần bằng **chấm điều hướng** bên phải
`products.html`	Tìm, lọc, sắp xếp 26 sản phẩm; yêu thích; **so sánh tối đa 3 mẫu**; xem chi tiết; bấm **Gửi báo giá** trên từng sản phẩm
`estimator.html`	Chọn loại, chất liệu, kích thước, lớp hoàn thiện, số lượng và xem giá ngay

Trang	Khách làm gì
`contact.html`	Gửi báo giá bằng đúng biểu mẫu đó; xem lịch sử yêu cầu và trạng thái gửi; **tải lịch sử dạng Excel**

Dùng chung mọi trang: menu (chỉ gồm các trang; các phần của trang chủ nằm ở chấm điều hướng cố định bên phải), **giỏ báo giá** (ngăn kéo bên phải), nút đổi ngôn ngữ **Việt / Anh**, chân trang có thông tin liên hệ, chính sách bảo mật, điều khoản sử dụng và hộp thoại GitHub.

---

2. Dự án gồm những gì

```
index.html  products.html  estimator.html  contact.html    4 trang
css/    style.css (nền tảng, biến màu/font)   features.css (các trang dữ liệu)   landing.css
(trang chủ)
js/     các module JavaScript (xem bên dưới)
data/   products.js  pricing.js  ui.js  config.js            bảng dữ liệu (cấu trúc JSON)
assets/ img/ (ảnh; p-*.jpg là ảnh sản phẩm từ Pexels, nguồn trong CREDITS.md), favicon.svg
apps-script/  Code.gs  (đoạn mã dán vào Google Apps Script để ghi Sheet)
docs/   HUONG-DAN-THUYET-TRINH.md (tài liệu này)   LOGIC-XU-LY.md (logic chi tiết)   slide báo
cáo .pptx
PROMPT_LOG.md   README.md   .github/workflows/deploy.yml
```

Các module JavaScript (mỗi tệp một việc)

Tệp	Làm gì	Nói ngắn gọn
`main.js`	Khởi động	Mỗi trang chỉ chạy phần nào có sẵn trên trang đó
`api.js`	Lấy bảng dữ liệu từ `Ericss.data`	"Đưa dữ liệu cho các module khác"
`store.js`	Đọc/ghi `localStorage` dạng JSON, không bao giờ làm sập trang	"Lưu dữ liệu của khách trên máy họ"
`pricing.js`	**Hàm tính tiền** (chiết khấu, mã giảm giá, VAT, ước giá, MOQ)	Hàm thuần: cùng đầu vào cho cùng kết quả, dễ test
`filters.js`	Tìm kiếm, lọc, sắp xếp sản phẩm	Hàm thuần, bỏ dấu tiếng Việt khi tìm
`validation.js`	Kiểm tra biểu mẫu	Hàm thuần
`cart.js` , `cart-ui.js`	Giỏ báo giá và giao diện của nó	

Tệp	Làm gì	Nói ngắn gọn
`catalog.js`	Trang sản phẩm: tìm, lọc, yêu thích, **so sánh 3 mẫu**	Chọn tối đa 3, bảng so sánh tô giá trị tốt nhất
`quote-form.js`	**Biểu mẫu báo giá duy nhất** dùng cho hộp thoại và trang Liên hệ	Một nơi sửa, hai nơi dùng
`quotes.js` , `sheet.js`	Tạo yêu cầu, lưu, gửi lên Google Sheet	
`history.js`	Lịch sử yêu cầu, xuất Excel (SheetJS)	Cột tiếng Việt giống Google Sheet
`lightbox.js`	Modal xem ảnh lớn ở trang chủ	Lật ảnh bằng nút, phím mũi tên, vuốt
`estimator.js`	Trang ước giá	
`site.js` , `site-i18n.js`	Menu, đổi ngôn ngữ, chấm điều hướng, hiệu ứng chung	

Chạy trên máy

Mở thẳng `index.html` bằng Chrome hoặc Edge là chạy, không cần máy chủ. Có thể dùng link GitHub Pages thay thế.

> Vì sao mở thẳng file được? Website dùng **script thường** và dữ liệu nằm trong `data/*.js` . Hai thứ trình duyệt chặn khi mở bằng `file://` là `fetch` đọc tệp `.json` và `import` / `export` của ES Modules, nên dự án không dùng chúng.

---

3. Theo từng tiêu chí rubric

Rubric có 5 tiêu chí (trong ảnh bạn gửi tổng là 90%). Mức cao nhất của mỗi tiêu chí được ghi sẵn, bên dưới là **bằng chứng trong dự án** và **cách nói**.

> Nếu bảng của giảng viên còn tiêu chí thứ 6 (ví dụ thuyết trình / báo cáo, 10%) mà ảnh không có, bạn hỏi lại để chuẩn bị thêm.

3.1. Giao diện UI/UX (20%)

Mức cao nhất: giao diện hiện đại, sáng tạo; trải nghiệm mượt; responsive hoàn hảo trên mọi thiết bị. Ghi chú trong rubric: "đúng chuẩn LHU (màu sắc/logo)".

Bằng chứng

- **Bố cục rõ ràng, nhất quán:** màu, font, khoảng cách khai báo một lần bằng **CSS Variables** ở `:root` trong `css/style.css` (`--color-primary` , `--font-body` , `--space-*` ...). Đổi một biến là đổi cả site.
- **Trang chủ:** hero có tiêu đề lớn và ảnh hình vòm, băng số liệu nền nâu, bốn chất liệu, quy trình 8 bước, thư viện ảnh có lọc, phần liên hệ. (Em đã thử hướng "tên là kiến trúc" bằng skill thiết kế rồi quay lại bản này vì thấy hợp hơn.)
- **Điều hướng tách bạch:** menu đầu trang chỉ chứa các **trang**; các **phần của trang chủ** là sáu chấm cố định ở mép phải, rê chuột hoặc Tab vào thì hiện nhãn, chấm của phần đang xem tự sáng lên.
- **Modal xem ảnh lớn** cho thư viện, quy trình, chất liệu, giới thiệu (nút trước/sau, phím mũi tên, vuốt).
- **Chân trang chuyên nghiệp:** bốn cột (giới thiệu, khám phá, liên hệ, pháp lý) với **Chính sách bảo mật** và **Điều khoản sử dụng** song ngữ, nói rõ thương hiệu giả định.
- **Responsive:** `Grid` và `Flexbox` , breakpoint ở 960px, 760px, 560px. Trên điện thoại: menu thu gọn, băng ảnh chất liệu thành lưới 2×2, dãy ảnh xưởng cuộn ngang bằng ngón tay. Đã kiểm tra không có cuộn ngang ở mọi trang.
- **Trải nghiệm:** thanh tiến trình cuộn, header đổi trạng thái khi cuộn, nút về đầu trang, hộp thoại và ngăn kéo có hiệu ứng, trạng thái "đang gửi", màn hình xác nhận thành công, thông báo lỗi ngay cạnh ô nhập.
- **Truy cập (accessibility):** liên kết "Chuyển đến nội dung", `aria-label` , focus nhìn thấy được, tôn trọng `prefers-reduced-motion` .
- **Thư viện hiệu ứng đúng đề:** Animate.css (tiêu đề, nút ở hero), AOS (các khối khi cuộn tới), GSAP (parallax ở hero, nút "từ tính"), vanilla-tilt (thẻ chất liệu nghiêng nhẹ).

Cách nói: "Em quản lý màu và font bằng biến CSS nên giao diện nhất quán. Menu chỉ chứa các trang, còn các phần của trang chủ có chấm điều hướng riêng nên không bị lẫn. Em kiểm tra responsive ở ba độ rộng 1440, 768 và 390."

Lưu ý "chuẩn LHU": website này mang thương hiệu riêng (Ericss) và có ghi tên Trường Đại học Lạc Hồng ở chân trang. Nếu giảng viên hỏi về "màu sắc/logo LHU", hãy hỏi lại rõ yêu cầu (đề tài chọn tự do hay phải theo nhận diện của trường). Đừng tự nhận là đã dùng màu của LHU.

3.2. Kỹ thuật JavaScript / DOM (30%) — tiêu chí nặng nhất

Mức cao nhất: tối ưu mã nguồn với `map` , `filter` ; xử lý sự kiện nâng cao; không có lỗi console; code chia module khoa học.

Đây là phần bạn cần hiểu kỹ nhất. Mỗi kỹ thuật dưới đây có ví dụ thật:

a) Duyệt mảng: `map` , `filter` , `reduce` , `forEach`

Kỹ thuật	Dùng để làm gì	Ở đâu
`filter`	Lọc sản phẩm theo chất liệu, giá, yêu thích, từ khóa	`js/filters.js` → `filterProducts` (nối nhiều `.filter()` liên tiếp)

Kỹ thuật	Dùng để làm gì	Ở đâu
`map`	Biến danh sách sản phẩm thành danh sách thẻ HTML	`js/catalog.js` → `visible.map(renderCard)`
`reduce`	Cộng tổng số lượng và tổng tiền của giỏ	`js/pricing.js` → `calcQuote` (`items.reduce(...)`)
`filter` + `reduce`	Chọn bậc chiết khấu cao nhất đã đạt	`js/pricing.js` → `volumeRate`
`sort`	Sắp xếp giá tăng/giảm, tên A–Z	`js/filters.js` → `sorters`
`flatMap`	Dựng các dòng "nhãn / giá trị" của bảng tổng	`js/cart-ui.js`
`forEach`	Gắn sự kiện cho nhiều nút, kiểm tra từng ô	`js/quote-form.js`, `js/site.js`
`Set` + `Math.min`	Tìm giá trị tốt nhất khi so sánh, và hàng nào các mẫu giống nhau	`js/catalog.js` (bảng so sánh)

Giải thích đơn giản để nói: *`map` biến mỗi phần tử thành thứ khác (sản phẩm → thẻ HTML); `filter` giữ lại phần tử thỏa điều kiện; `reduce` gom cả mảng về một giá trị (cộng ra tổng tiền).*

b) Thao tác DOM linh hoạt

- Hàm `h('div', {class: '...'}, con...)` trong `js/ui.js` tạo phần tử HTML bằng `document.createElement`, thay cho việc nối chuỗi HTML (an toàn hơn, tránh chèn mã độc từ dữ liệu).
- `replaceChildren(...)` vẽ lại cả danh sách sau mỗi lần lọc.
- Chuyển ngôn ngữ: cập nhật `textContent` / `alt` / `aria-label` của hàng trăm phần tử theo khóa dịch (`data-t`, `data-i18n`).

c) Xử lý sự kiện (click, input, change, submit...)

- `input` ở ô tìm kiếm kèm **debounce** (`js/ui.js`): chờ khách ngừng gõ 150 ms rồi mới lọc, tránh vẽ lại liên tục.
- `submit` với `event.preventDefault()` để tự kiểm tra và gửi, không tải lại trang (`js/quote-form.js`).
- **Event delegation** (ủy quyền sự kiện): thay vì gắn sự kiện cho từng nút của 26 thẻ, gắn **một** lắng nghe ở khung chứa rồi dùng `event.target.closest('[data-action]')` để biết nút nào được bấm (`js/catalog.js`, `js/cart-ui.js`). Nhanh hơn và vẫn chạy với thẻ được vẽ thêm về sau.
- Bàn phím và cảm ứng: Esc đóng hộp thoại/menu; ở modal ảnh có phím mũi tên và vuốt ngang (`touchstart` / `touchend`); ảnh vào được bằng Tab và Enter (`js/lightbox.js`).
- `IntersectionObserver` (đếm số khi cuộn tới, đánh dấu chấm điều hướng của phần đang xem); `requestAnimationFrame` (thanh tiến trình).

d) Bất đồng bộ và xử lý lỗi

- `async/await` + `fetch` để gửi yêu cầu lên Google Sheet (`js/sheet.js`); dữ liệu sản phẩm đã có sẵn trong trang nên không cần chờ tải.
- `try/catch` ở mọi chỗ có thể hỏng: `localStorage` bị chặn, dữ liệu nhập vào là JSON sai, mạng mất khi gửi Google Sheet. Khi lỗi, trang không sập: yêu cầu vẫn được lưu và ghi trạng thái "chờ đồng bộ".

e) Chia module khoa học

- Mỗi tệp là một **module kiểu IIFE** (hàm tự gọi, biến bên trong không lọt ra ngoài) và chỉ "xuất" những hàm cần thiết vào đối tượng chung `Ericss` (ví dụ `Ericss.pricing.calcQuote`); mỗi tệp một trách nhiệm (bảng ở mục 2). Các trang nạp tệp theo thứ tự phụ thuộc.
- **Tách hàm thuần khỏi giao diện:** `pricing.js` , `filters.js` , `validation.js` không đụng DOM nên đọc, kiểm tra và tái sử dụng độc lập rất dễ.
- Một biểu mẫu báo giá duy nhất (`quote-form.js`) thay vì hai bản sao: sửa một nơi, áp dụng cho cả hộp thoại và trang Liên hệ.

f) Không lỗi console: đã kiểm tra tự động bằng trình duyệt thật ở mọi trang (không có lỗi hay request thất bại).

Cách nói: "Phần cốt lõi em tách thành hàm thuần, không đụng giao diện: lọc dùng nối `filter` , giỏ hàng dùng `reduce` , danh sách thẻ dùng `map` . Với 26 thẻ em dùng event delegation để chỉ có một trình lắng nghe."

3.3. Logic dữ liệu JSON (15%)

Mức cao nhất: cấu trúc dữ liệu tối ưu, bao quát mọi trường hợp; các hàm tính toán (GPA, học phí, tiền...) chính xác tuyệt đối.

Cấu trúc JSON (mở `data/products.js` ra chỉ):

```
{
  "id": "rope-corde-chair",       // khóa duy nhất, dùng trong URL (?product=...) và giỏ hàng
  "sku": "ERC-RP-001",            // mã hàng: ERC-<chất liệu>-<số thứ tự>
  "category": "rope",
  "name": { "vi": "...", "en": "..." },          // song ngữ ngay trong dữ liệu
  "description": { "vi": "...", "en": "..." },
  "price": 6800000,               // số nguyên (đồng), không dùng số thập phân để tránh sai số
  "moq": 20,                      // MOQ của xưởng: số lượng đặt tối thiểu
  "dimensions": "58 × 60 × 82 cm",
  "leadTimeDays": 22,
  "image": "assets/img/g-rope-chair.jpg",
  "tags": ["chair", "dining", "handmade"]
}
```

- `data/pricing.js` : `vatRate` , `volumeDiscounts` (bậc 5 / 10 / 20 sản phẩm → 3% / 5% / 8%), `coupons` (mã, tỷ lệ, đơn tối thiểu), `estimator` (bảng giá theo loại, chất liệu, kích thước, lớp hoàn thiện).

- Tách dữ liệu khỏi mã: muốn đổi giá hay thêm sản phẩm chỉ sửa JSON, không động tới JavaScript.

Công thức tính tiền (`js/pricing.js` → `calcQuote`), theo đúng thứ tự:

1. Tạm tính = Σ (đơn giá × số lượng)
2. Chiết khấu số lượng = tạm tính × tỷ lệ của bậc cao nhất đã đạt
3. Mã giảm giá = (tạm tính − chiết khấu) × tỷ lệ mã, **chỉ khi** tạm tính đạt mức tối thiểu của mã
4. VAT = (sau chiết khấu và mã) × 10%
5. Tổng = sau chiết khấu và mã + VAT. Mọi số làm tròn đến đồng.

Ví dụ tính tay để trình bày (bạn tự bấm trên trang Sản phẩm: ghế dây thừng Cordé, số lượng 50, sẽ ra đúng con số này):

- 50 ghế × 6.800.000 ₫ = **340.000.000 ₫**
- Đặt 50 (≥ 20) nên giảm 8% = 27.200.000 → còn 312.800.000 ₫
- VAT 10% = 31.280.000 → **Tổng 344.080.000 ₫**

Các trường hợp biên đã xử lý (nói để ăn điểm "bao quát mọi trường hợp")

- Giỏ trống → tổng 0.
- Mã sai → báo "không hợp lệ"; mã chưa đủ đơn tối thiểu → báo "cần đơn từ ..." và không áp dụng; gõ hoa/thường hay có khoảng trắng đều nhận.
- **Sai số số thực:** `14.662.499,999…` thay vì `14.662.500` làm giá lệch 1.000 ₫. Đã sửa bằng `toPrecision(12)` trước khi làm tròn.
- Tìm kiếm không phân biệt dấu tiếng Việt: gõ "day thung" ra "dây thừng", cả chữ "Đ" hoa.
- MOQ khách nhập thấp hơn MOQ xưởng → vẫn cho gửi nhưng cảnh báo "giá có thể thay đổi".
- Số lượng không hợp lệ (0, âm, số lẻ, chữ, quá giới hạn) → báo lỗi.

Cách nói: "Dữ liệu em để trong JSON, giá lưu bằng số nguyên đồng để không bị sai số thập phân. Hàm tính tiền tách riêng nên em kiểm tra được bằng số tính tay, và em đã bắt được lỗi làm tròn ở biên .5."

3.4. Quy trình Agile / Git (15%)

Mức cao nhất: dùng GitHub Projects/Kanban chuyên nghiệp; lịch sử commit đều đặn từ đa số thành viên, thể hiện phối hợp chặt chẽ của cả nhóm.

> **Đây là phần bạn phải tự làm, tôi (AI) không làm thay được và bạn cũng không nên làm giả.**
> Rubric chấm cả **Kanban** và **số thành viên có commit**. Lịch sử commit hiện tại chỉ có tài khoản của bạn.

Đã có sẵn

- Commit rõ ràng, bằng tiếng Việt, theo tiền tố chuẩn: `feat:` (tính năng), `fix:` (sửa lỗi), `docs:` (tài liệu), `refactor:` (dọn mã), `ci:` (tự động hóa). Xem bằng `git log --oneline` .

- Tự động deploy: mỗi lần push lên `main`, GitHub Actions đưa website lên GitHub Pages (`.github/workflows/deploy.yml`).

Bạn cần làm cho nhóm

1. Tạo **GitHub Projects** (loại Board / Kanban) với các cột *Backlog → Đang làm → Review → Xong*.
2. Mỗi tính năng là một **issue** (thẻ), gán cho một thành viên. Gợi ý chia việc theo module có sẵn: - Giao diện trang chủ (`landing.css`, `index.html`) - Danh sách sản phẩm và bộ lọc (`catalog.js`, `filters.js`) - Giỏ báo giá và tính tiền (`cart.js`, `pricing.js`) - Biểu mẫu báo giá và Google Sheet (`quote-form.js`, `sheet.js`, `apps-script/`) - Trang ước giá (`estimator.js`) và tài liệu
3. Mỗi người làm trên **nhánh riêng**, mở **Pull Request**, người khác review rồi mới gộp vào `main`. Như vậy lịch sử có commit của nhiều người, đều đặn theo thời gian.
4. Cập nhật thẻ Kanban khi làm (kéo sang cột tiếp theo) để bảng "cập nhật đều".

Cách nói (nếu làm nhóm): "Chúng em chia việc theo module, mỗi người một nhánh, review chéo bằng Pull Request, tiến độ theo dõi trên bảng Kanban."

3.5. Vibe Coding với AI (10%)

Mức cao nhất: README chuyên nghiệp; chứng minh tư duy điều khiển AI qua các prompt logic để xử lý vấn đề khó.

Bằng chứng

- `README.md` có mô tả dự án, kiến trúc, cách chạy, công thức, và **bảng "Các prompt tiêu biểu"** (prompt, vấn đề, cách điều khiển AI).
- `PROMPT_LOG.md` ghi 28 vòng làm việc, mỗi vòng có: prompt đã dùng, AI tạo ra gì, thách thức gặp phải, cách sửa prompt.

Những ví dụ "điều khiển AI" mạnh nhất, nên kể

1. **AI làm sai, mình bắt được bằng số tính tay:** AI viết hàm làm tròn tiền, em đối chiếu với số tính tay và lộ lỗi biên .5 → sửa bằng `toPrecision(12)`. *Bài học: đừng tin kết quả AI, hãy kiểm chứng.*
2. **Chẩn đoán đúng nguyên nhân:** hiệu ứng mở ảnh không chạy; AI đoán lung tung, em chỉ ra `IntersectionObserver` coi phần tử bị `clip-path` cắt hết là "không nhìn thấy" → quan sát phần tử cha.
3. **Ràng buộc kiến trúc:** yêu cầu AI không dùng `<div>` vô nghĩa; gộp hai biểu mẫu thành một module thay vì sao chép.
4. **Hạn chế của AI:** AI không thể tự triển khai Google Apps Script thay em (cần tài khoản Google), nên em yêu cầu AI viết sẵn mã và hướng dẫn, em tự triển khai.
5. **Biết khi nào AI nên từ chối:** khi em muốn lấy ảnh từ website bán hàng khác, AI từ chối vì ảnh có bản quyền và đề xuất ảnh miễn phí; kết quả là ảnh sản phẩm lấy từ Pexels, có ghi nguồn.
6. **Dùng công cụ để kiểm soát chất lượng:** cài skill thiết kế, để công cụ đo trang trong trình duyệt thật (cỡ chữ, đường kẻ, màu, độ phủ) rồi sửa tới khi hết lỗi, thay vì "cảm tính".

Cách nói: "AI giúp em nhanh ở khung HTML/CSS và các đoạn lặp, nhưng logic tiền bạc em luôn kiểm bằng số tính tay. Em coi AI như một lập trình viên cần được review."

---

4. Kịch bản demo (khoảng 6 phút)

Bước	Làm gì	Nói gì
1	Mở trang chủ, kéo xuống; rê chuột vào các chấm bên phải rồi bấm một chấm	Menu chỉ chứa trang, các phần có chấm điều hướng riêng với nhãn
2	Lọc thư viện ảnh theo chất liệu rồi bấm một ảnh	Modal xem ảnh lớn, lật bằng mũi tên; chỉ lật qua các ảnh đang lọc
3	Bấm EN rồi VI	Song ngữ, nhớ lựa chọn, tiền tệ đổi định dạng theo ngôn ngữ
4	Vào Sản phẩm: gõ "day thung", lọc Teak, kéo thanh giá, sắp xếp	`filter` nối tiếp; tìm không dấu; "Xem thêm" phân trang
5	Bấm tim một sản phẩm, tải lại trang; tick "So sánh" ở 3 mẫu rồi bấm So sánh	`localStorage` lưu yêu thích; bảng so sánh tô giá thấp nhất, MOQ thấp nhất, giao nhanh nhất
6	**Gửi báo giá** ở một sản phẩm; nhập số lượng thấp hơn MOQ rồi bấm gợi ý ×2, ×5	Cảnh báo MOQ; chiết khấu tự đổi; tạm tính cập nhật tức thì
7	Thêm vài món vào giỏ, nhập mã `ERICSS10`	`reduce`; mã chỉ áp dụng khi đủ điều kiện
8	Sang Liên hệ: gửi khi để trống (cho thấy lỗi), rồi gửi đúng	Kiểm tra form; **cùng một biểu mẫu** với hộp thoại
9	Sau khi gửi: chỉ ra thông báo "đã nhận, sẽ liên hệ sớm nhất" và **mã QR**; mở Google Sheet cho thấy dòng mới (cột MOQ xưởng, MOQ khách đặt, Đạt MOQ, Tổng cộng); mở email cảm ơn của khách	Cầu nối Apps Script; lưu cục bộ trước, mất mạng vẫn không mất yêu cầu
10	Mở chân trang, bấm Chính sách bảo mật; tải Excel lịch sử ở trang Liên hệ	Pháp lý khớp với cách xử lý dữ liệu thật; Excel có cột tiếng Việt
11	Mở `js/pricing.js` và chỉ hàm `calcQuote`; mở tab Actions trên GitHub	Công thức đúng thứ tự; push là tự deploy
12	Mở `PROMPT_LOG.md` và bảng prompt trong README	Tư duy điều khiển AI

Mẹo: trước khi demo, mở sẵn các tab và gửi thử một yêu cầu để chắc Sheet đang nhận. Chạy bằng link GitHub Pages (hoặc mở thẳng `index.html`). Khi tự thử gửi nhiều lần sẽ gặp giới hạn spam (3 yêu cầu mỗi email mỗi giờ, nghỉ 30 giây giữa hai lần), nên dùng email khác nhau.

5. Câu hỏi giảng viên có thể hỏi

Vì sao dữ liệu nằm riêng trong `data/*.js` mà không viết thẳng vào mã xử lý? Tách dữ liệu khỏi logic: đổi giá hay thêm sản phẩm chỉ sửa bảng dữ liệu, không động tới mã tính toán. Nội dung bảng vẫn là cấu trúc JSON; đặt trong tệp `.js` để trang mở thẳng từ ổ đĩa được (trình duyệt không cho `fetch` đọc `.json` ở chế độ `file://`). Nếu giảng viên đòi tệp `.json` thật, bạn trả lời được rõ lý do này.

Dữ liệu khách lưu ở đâu? Mất khi nào? Giỏ, yêu thích, lịch sử lưu ở `localStorage` của trình duyệt, nên theo từng máy và sẽ mất khi khách xóa dữ liệu trình duyệt. Yêu cầu báo giá thì **gửi thêm lên Google Sheet** nên không mất.

Không có máy chủ thì ghi Google Sheet bằng cách nào? Dùng Google Apps Script làm "web app" nhận `POST` rồi ghi dòng vào Sheet (`apps-script/Code.gs`).

Email cảm ơn gửi bằng cách nào? Có bị lạm dụng không? Apps Script dùng `MailApp.sendEmail` gửi từ tài khoản Google của chủ website, ngay sau khi ghi dòng vào Sheet. Vì link web app công khai nên em chống lạm dụng: mỗi email chỉ gửi được tối đa 3 yêu cầu mỗi giờ (đếm bằng `CacheService`) nên cũng chỉ nhận tối đa 3 thư, dừng khi hạn mức thư trong ngày sắp hết, chỉ gửi nội dung khách đã nhập và có câu "nếu không phải bạn, hãy bỏ qua".

Mã QR lấy ở đâu, chứa gì? Gọi API QuickChart (`quickchart.io/qr?text=...`) với nội dung là họ tên và mã yêu cầu; ảnh hiện ngay trong màn hình xác nhận và nằm trong email. Cần nói thật: họ tên và mã được gửi tới quickchart.io để vẽ ảnh.

Làm sao chặn spam báo giá? Em chống spam ở hai lớp. **Trên trình duyệt:** ô ẩn (honeypot), thời gian điền form được gửi kèm (`fillMs`), mỗi thiết bị phải chờ 30 giây giữa hai yêu cầu và tối đa 5 yêu cầu mỗi giờ. **Trong Apps Script** (lớp quyết định, vì khách có thể bỏ qua trình duyệt): từ chối form điền dưới 3 giây, tên quá ngắn, email sai định dạng, nội dung quá dài hoặc có hơn 2 đường link; mỗi email tối đa 3 yêu cầu mỗi giờ, toàn hệ thống tối đa 6 yêu cầu mỗi phút và 60 mỗi giờ. Yêu cầu bị từ chối không ghi dòng nào, không gửi email. Phần quyết định nằm ở Apps Script vì ai cũng có thể bỏ qua trình duyệt và gọi thẳng link; các bộ đếm chạy trong khóa `LockService` nên gửi đồng thời cũng không vượt giới hạn.

Làm sao tránh ghi trùng một yêu cầu? Mỗi yêu cầu có `id` duy nhất; script kiểm tra `id` đã có trong Sheet thì bỏ qua. Nhờ đó website gửi lại nhiều lần cũng không trùng.

Nếu mất mạng lúc gửi thì sao? Yêu cầu đã lưu cục bộ ở trạng thái "chờ đồng bộ", tự gửi lại khi tải trang hoặc bấm "Gửi lại".

Có chống "công thức độc hại" trong Google Sheet không? Có. Script thêm dấu `'` phía trước ô bắt đầu bằng `=` , `+` , `-` , `@` để Google Sheet không coi là công thức (chống formula injection). Chặn spam xem câu "Làm sao chặn spam báo giá?" ở trên.

Chặn spam có hoàn hảo không? Không, và em nói rõ giới hạn: Apps Script không biết địa chỉ IP nên không chặn theo IP được; giới hạn theo email có thể bị né bằng nhiều email, nên em thêm trần toàn hệ thống (6 mỗi phút, 60 mỗi giờ). Kẻ gọi thẳng link có thể bỏ trường `fillMs` . Với hệ thống thật cần thêm CAPTCHA hoặc máy chủ riêng chặn theo IP. Chi tiết ở `LOGIC-XU-LY.md` , mục 3.

So sánh sản phẩm hoạt động thế nào? Chọn tối đa 3 sản phẩm bằng ô "So sánh" (lưu `localStorage`), bảng so sánh có 6 hàng; ba hàng giá, MOQ, thời gian giao được tô xanh ở mẫu tốt nhất, chỉ khi các mẫu thật sự khác nhau. Có công tắc "Chỉ hiện điểm khác nhau".

Ảnh sản phẩm lấy ở đâu? Có vấn đề bản quyền không? Ảnh sản phẩm `p-*.jpg` lấy từ Pexels (giấy phép cho phép dùng miễn phí), có ghi nguồn trong `assets/img/CREDITS.md` và trong Điều khoản sử dụng. Đây là ảnh minh họa kiểu dáng, không phải sản phẩm thật. Em không lấy ảnh từ website bán hàng khác vì ảnh đó có bản quyền.

Event delegation là gì, vì sao dùng? Gắn một lắng nghe ở phần tử cha thay vì từng phần tử con; dùng `event.target.closest()` để biết con nào được bấm. Nhẹ hơn và chạy cả với phần tử vẽ thêm sau.

`map`, `filter`, `reduce` khác nhau thế nào? (xem 3.2a)

Debounce là gì? Chờ người dùng ngừng thao tác một khoảng ngắn mới chạy, để không lọc lại sau mỗi phím.

Hàm thuần là gì, vì sao dễ kiểm tra? Cùng đầu vào luôn cho cùng kết quả, không đụng DOM hay lưu trữ. Nên chỉ cần gọi hàm với số và so sánh kết quả.

ES Module khác script thường chỗ nào, vì sao em chọn script thường? ES Module có `import` / `export` và phạm vi riêng cho từng tệp, nhưng không chạy được khi mở bằng `file://`. Em dùng script thường và tự tạo phạm vi riêng bằng IIFE, chỉ đưa ra những hàm cần dùng qua `Ericss`, nên vẫn tránh trùng tên biến toàn cục mà mở thẳng file là chạy. Đổi lại phải nạp tệp đúng thứ tự (em ghi rõ ở cuối mỗi trang).

MOQ là gì, hệ thống xử lý ra sao? MOQ là số lượng đặt tối thiểu. Mỗi sản phẩm có MOQ của xưởng; khách nhập số lượng của họ, hệ thống báo đạt hay chưa và lưu cả hai con số cùng cột `meetsMoq` vào Sheet.

Vì sao giá lưu bằng số nguyên? Số thập phân trong máy tính có sai số (ví dụ `0.1 + 0.2 ≠ 0.3`). Dùng số nguyên đồng và làm tròn có kiểm soát để tiền luôn chính xác.

Semantic HTML ở đâu? `header`, `nav`, `main`, `section` (có `aria-labelledby`), `article`, `figure`, `dl` (con số), `ol` (quy trình, vì thứ tự quan trọng), `dialog`, `footer`, `address`.

Hiệu ứng có làm khó người dùng nhạy cảm chuyển động không? Có `prefers-reduced-motion`: khi bật, các hiệu ứng chuyển động tự tắt, nội dung vẫn đầy đủ.

Bạn dùng AI ở đâu, và bạn tự hiểu code chứ? Xem 3.5. Bạn nên tự đọc `pricing.js`, `filters.js`, `quote-form.js` và giải thích được từng hàm.

---

6. Phải trung thực khi thuyết trình

- Số liệu "thành lập 2008, 120 nghệ nhân, 15.000 m²", tên Ericss và dữ liệu 26 sản phẩm là **nội dung mẫu** cho bài làm, không phải doanh nghiệp thật.

- Ảnh sản phẩm `p-*.jpg` (12 ảnh) lấy từ Pexels, có nguồn trong `assets/img/CREDITS.md`; 26 sản phẩm dùng 19 ảnh khác nhau, riêng sản phẩm dây thừng vẫn dùng ảnh cũ. Ảnh xưởng, quy trình và vật liệu là ảnh minh họa cho mục đích học tập: nếu cần bản quyền rõ ràng, thay bằng ảnh bạn có quyền dùng.
- Website ghi rõ ở chân trang và Điều khoản rằng **thương hiệu là giả định, không bán hàng, không nhận thanh toán**; phần pháp lý được viết đúng với cách website thật sự xử lý dữ liệu, không có mã số thuế hay giấy phép bịa ra.
- **Hạn chế đã biết:** `localStorage` là theo từng máy; đường dẫn Google Apps Script nằm trong mã công khai nên ai có link cũng gửi được (đã có chặn spam hai lớp nhưng không chặn theo IP được; hệ thống thật cần xác thực hoặc CAPTCHA); chưa có trang quản trị để xem yêu cầu (xem trong Google Sheet); mã QR được vẽ nhờ quickchart.io nên họ tên và mã yêu cầu được gửi tới dịch vụ đó.
- Giao diện do AI dựng dưới sự hướng dẫn của bạn và đã đổi hướng vài lần theo yêu cầu (thử thiết kế "tên là kiến trúc" rồi quay lại hero hình vòm; đổi menu thành chấm điều hướng); bạn nên nắm được lý do mỗi lần đổi.

7. Danh sách kiểm tra trước giờ thuyết trình

- ☐ Mở link GitHub Pages, thử cả 4 trang và đổi ngôn ngữ
- ☐ Gửi thử một yêu cầu, thấy dòng mới trong Google Sheet (xóa dòng thử sau đó)
- ☐ Tự bấm một ví dụ trên trang Sản phẩm và đối chiếu với số tính tay trong mục 3.3
- ☐ Tab **Actions** trên GitHub có dấu xanh
- ☐ Bảng **Kanban** có thẻ và có commit của các thành viên (xem 3.4)
- ☐ Đọc lại `PROMPT_LOG.md` để kể được ít nhất 3 lần bạn sửa AI
- ☐ Tự giải thích được `calcQuote`, `filterProducts` và `quote-form.js`
- ☐ Thử so sánh 3 sản phẩm, modal xem ảnh và chấm điều hướng ở trang chủ
- ☐ Đọc `LOGIC-XU-LY.md` mục 3 (chặn spam) để trả lời được câu hỏi vặn về giới hạn